

EVALUATION AND QUALITY ASSURANCE

Module 6 — Governed AI Feature Delivery

THE PROBLEM WE'RE SOLVING

Without evaluation, quality discussions become opinion-based.

-
-
-
-

THE PROBLEM WE'RE SOLVING

Without evaluation, quality discussions become opinion-based.

- No baseline for model or prompt changes
-
-
-

THE PROBLEM WE'RE SOLVING

Without evaluation, quality discussions become opinion-based.

- No baseline for model or prompt changes
- Bugs discovered only in production
-
-

THE PROBLEM WE'RE SOLVING

Without evaluation, quality discussions become opinion-based.

- No baseline for model or prompt changes
- Bugs discovered only in production
- No shared quality criteria across teams
-

THE PROBLEM WE'RE SOLVING

Without evaluation, quality discussions become opinion-based.

- No baseline for model or prompt changes
- Bugs discovered only in production
- No shared quality criteria across teams
- Weak release confidence

THE PROBLEM WE'RE SOLVING

Without evaluation, quality discussions become opinion-based.

- No baseline for model or prompt changes
- Bugs discovered only in production
- No shared quality criteria across teams
- Weak release confidence

Now: define lightweight, repeatable AI evals.

WHY EVALS MATTER IN TEAMS

-
-
-
-

WHY EVALS MATTER IN TEAMS

- They replace opinion with measurable quality signals.
-
-
-

WHY EVALS MATTER IN TEAMS

- They replace opinion with measurable quality signals.
- They make prompt and model changes safer and faster to adopt.
-
-

WHY EVALS MATTER IN TEAMS

- They replace opinion with measurable quality signals.
- They make prompt and model changes safer and faster to adopt.
- They provide shared release criteria across teams.
-

WHY EVALS MATTER IN TEAMS

- They replace opinion with measurable quality signals.
- They make prompt and model changes safer and faster to adopt.
- They provide shared release criteria across teams.
- They create audit-ready evidence of quality decisions.

WHY EVALS MATTER IN TEAMS

- They replace opinion with measurable quality signals.
- They make prompt and model changes safer and faster to adopt.
- They provide shared release criteria across teams.
- They create audit-ready evidence of quality decisions.

Evals are not research infrastructure. They are a delivery discipline.

WHAT TO EVALUATE

-
-
-
-

WHAT TO EVALUATE

- Correctness and schema compliance
-
-
-

WHAT TO EVALUATE

- Correctness and schema compliance
- Safety and refusal behaviour
-
-

WHAT TO EVALUATE

- Correctness and schema compliance
- Safety and refusal behaviour
- Hallucination and grounding checks
-

WHAT TO EVALUATE

- Correctness and schema compliance
- Safety and refusal behaviour
- Hallucination and grounding checks
- Latency and cost thresholds

WHAT TO EVALUATE

- Correctness and schema compliance
- Safety and refusal behaviour
- Hallucination and grounding checks
- Latency and cost thresholds

If it matters for release, it needs a measurable criterion.

MANUAL QA VS REPEATABLE EVALS

Approach	Strength	Limitation
Manual QA	Fast spot-checking	Not scalable, inconsistent coverage
Repeatable eval suite	Reliable regression detection	Requires initial dataset setup

MANUAL QA VS REPEATABLE EVALS

Approach	Strength	Limitation
Manual QA	Fast spot-checking	Not scalable, inconsistent coverage
Repeatable eval suite	Reliable regression detection	Requires initial dataset setup

Use both. Never rely on manual QA alone for release decisions.

EVALUATION DIMENSIONS

- 1.
- 2.
- 3.
- 4.
- 5.

EVALUATION DIMENSIONS

1. Correctness: is output factually and functionally right?
- 2.
- 3.
- 4.
- 5.

EVALUATION DIMENSIONS

1. Correctness: is output factually and functionally right?
2. Format: does output match the schema and contract?
- 3.
- 4.
- 5.

EVALUATION DIMENSIONS

1. Correctness: is output factually and functionally right?
2. Format: does output match the schema and contract?
3. Safety: does behaviour follow policy and refusal rules?
- 4.
- 5.

EVALUATION DIMENSIONS

1. Correctness: is output factually and functionally right?
2. Format: does output match the schema and contract?
3. Safety: does behaviour follow policy and refusal rules?
4. Grounding: is the response supported by input or evidence?
- 5.

EVALUATION DIMENSIONS

1. Correctness: is output factually and functionally right?
2. Format: does output match the schema and contract?
3. Safety: does behaviour follow policy and refusal rules?
4. Grounding: is the response supported by input or evidence?
5. Operational: latency, cost, and error-rate targets.

EVALUATION DIMENSIONS

1. Correctness: is output factually and functionally right?
2. Format: does output match the schema and contract?
3. Safety: does behaviour follow policy and refusal rules?
4. Grounding: is the response supported by input or evidence?
5. Operational: latency, cost, and error-rate targets.

A release decision based on one dimension alone is incomplete.

GOLDEN DATASETS

-
-
-
-

GOLDEN DATASETS

- Curated input and output cases tied to real workflows.
-
-
-

GOLDEN DATASETS

- Curated input and output cases tied to real workflows.
- Include both happy-path cases and failure edge cases.
-
-

GOLDEN DATASETS

- Curated input and output cases tied to real workflows.
- Include both happy-path cases and failure edge cases.
- Version the dataset alongside prompt and model changes.
-

GOLDEN DATASETS

- Curated input and output cases tied to real workflows.
- Include both happy-path cases and failure edge cases.
- Version the dataset alongside prompt and model changes.
- Start small and representative rather than large and shallow.

GOLDEN DATASETS

- Curated input and output cases tied to real workflows.
- Include both happy-path cases and failure edge cases.
- Version the dataset alongside prompt and model changes.
- Start small and representative rather than large and shallow.

Ten meaningful cases beat one hundred vague ones.

DESIGNING PASS/FAIL CRITERIA

-
-
-
-

DESIGNING PASS/FAIL CRITERIA

- Define machine-checkable expectations where possible.
-
-
-

DESIGNING PASS/FAIL CRITERIA

- Define machine-checkable expectations where possible.
- Separate hard failures from soft quality thresholds.
-
-

DESIGNING PASS/FAIL CRITERIA

- Define machine-checkable expectations where possible.
- Separate hard failures from soft quality thresholds.
- Use explicit tolerances for numeric and format variance.
-

DESIGNING PASS/FAIL CRITERIA

- Define machine-checkable expectations where possible.
- Separate hard failures from soft quality thresholds.
- Use explicit tolerances for numeric and format variance.
- Map each criterion to release risk, not just technical correctness.

EXAMPLE EVAL CASE: EXTRACTION

Field	Expected	Rule
documentType	invoice	Exact match required
confidence	0..1	Must be within valid range
entities	Includes amount and invoice_number	Required keys must be present
Refusal behaviour	needs_review on low confidence	Policy-compliant fallback required

EXAMPLE EVAL CASE: EXTRACTION

Field	Expected	Rule
documentType	invoice	Exact match required
confidence	0..1	Must be within valid range
entities	Includes amount and invoice_number	Required keys must be present
Refusal behaviour	needs_review on low confidence	Policy-compliant fallback required

Each rule maps directly to a release risk. If this case fails, the feature should not ship.

PROMPT AND MODEL COMPARISON WORKFLOW

-
-
-
-

PROMPT AND MODEL COMPARISON WORKFLOW

- Run the same dataset across all variants being compared.
-
-
-

PROMPT AND MODEL COMPARISON WORKFLOW

- Run the same dataset across all variants being compared.
- Compare accuracy, safety, latency, and cost together.
-
-

PROMPT AND MODEL COMPARISON WORKFLOW

- Run the same dataset across all variants being compared.
- Compare accuracy, safety, latency, and cost together.
- Promote a variant only if quality improves or remains within acceptable bounds.
-

PROMPT AND MODEL COMPARISON WORKFLOW

- Run the same dataset across all variants being compared.
- Compare accuracy, safety, latency, and cost together.
- Promote a variant only if quality improves or remains within acceptable bounds.
- Record the rationale for the chosen variant as part of the release record.

TRACE-DRIVEN DEBUGGING

-
-
-
-

TRACE-DRIVEN DEBUGGING

- Inspect failed eval traces to find the exact failure stage.
-
-
-

TRACE-DRIVEN DEBUGGING

- Inspect failed eval traces to find the exact failure stage.
- Distinguish prompt issues from parsing issues from policy gate failures.
-
-

TRACE-DRIVEN DEBUGGING

- Inspect failed eval traces to find the exact failure stage.
- Distinguish prompt issues from parsing issues from policy gate failures.
- Turn recurring failures into permanent test cases in the golden dataset.
-

TRACE-DRIVEN DEBUGGING

- Inspect failed eval traces to find the exact failure stage.
- Distinguish prompt issues from parsing issues from policy gate failures.
- Turn recurring failures into permanent test cases in the golden dataset.
- Use traces to explain release decisions clearly to stakeholders and auditors.

LIGHTWEIGHT EVAL ARCHITECTURE

-
-
-
-

LIGHTWEIGHT EVAL ARCHITECTURE

- Dataset files, a runner, a scorer, and a report output.
-
-
-

LIGHTWEIGHT EVAL ARCHITECTURE

- Dataset files, a runner, a scorer, and a report output.
- Start local, then integrate into CI as a release gate.
-
-

LIGHTWEIGHT EVAL ARCHITECTURE

- Dataset files, a runner, a scorer, and a report output.
- Start local, then integrate into CI as a release gate.
- Keep the framework simple enough for the product team to own and extend.
-

LIGHTWEIGHT EVAL ARCHITECTURE

- Dataset files, a runner, a scorer, and a report output.
- Start local, then integrate into CI as a release gate.
- Keep the framework simple enough for the product team to own and extend.
- Do not over-engineer before the baseline is stable.

LIGHTWEIGHT EVAL ARCHITECTURE

- Dataset files, a runner, a scorer, and a report output.
- Start local, then integrate into CI as a release gate.
- Keep the framework simple enough for the product team to own and extend.
- Do not over-engineer before the baseline is stable.

A simple eval that runs consistently is more valuable than a sophisticated one that does not.

COMMON EVAL ANTI-PATTERNS

-
-
-
-

COMMON EVAL ANTI-PATTERNS

- Measuring only one dimension, such as accuracy alone.
-
-
-

COMMON EVAL ANTI-PATTERNS

- Measuring only one dimension, such as accuracy alone.
- No safety or refusal checks in the eval suite.
-
-

COMMON EVAL ANTI-PATTERNS

- Measuring only one dimension, such as accuracy alone.
- No safety or refusal checks in the eval suite.
- Dataset drift without version tracking alongside prompts and models.
-

COMMON EVAL ANTI-PATTERNS

- Measuring only one dimension, such as accuracy alone.
- No safety or refusal checks in the eval suite.
- Dataset drift without version tracking alongside prompts and models.
- Ignoring latency and cost impact when comparing quality improvements.

MODULE 6 LAB BUILD TARGET

You will create a lightweight eval suite with:

-
-
-
-
-

MODULE 6 LAB BUILD TARGET

You will create a lightweight eval suite with:

- A golden dataset of inputs with expected outputs
-
-
-
-

MODULE 6 LAB BUILD TARGET

You will create a lightweight eval suite with:

- A golden dataset of inputs with expected outputs
- Machine-checkable pass/fail rules per case
-
-
-

MODULE 6 LAB BUILD TARGET

You will create a lightweight eval suite with:

- A golden dataset of inputs with expected outputs
- Machine-checkable pass/fail rules per case
- A prompt or model comparison run on the same dataset
-
-

MODULE 6 LAB BUILD TARGET

You will create a lightweight eval suite with:

- A golden dataset of inputs with expected outputs
- Machine-checkable pass/fail rules per case
- A prompt or model comparison run on the same dataset
- Trace inspection notes for any failed cases
-

MODULE 6 LAB BUILD TARGET

You will create a lightweight eval suite with:

- A golden dataset of inputs with expected outputs
- Machine-checkable pass/fail rules per case
- A prompt or model comparison run on the same dataset
- Trace inspection notes for any failed cases
- A release recommendation with written rationale

MODULE 6 LAB BUILD TARGET

You will create a lightweight eval suite with:

- A golden dataset of inputs with expected outputs
- Machine-checkable pass/fail rules per case
- A prompt or model comparison run on the same dataset
- Trace inspection notes for any failed cases
- A release recommendation with written rationale

Definition of done: the suite runs without manual intervention and produces a clear pass or fail result per case.

SUMMARY

- 1.
- 2.
- 3.
- 4.

SUMMARY

1. **If it matters, evaluate it** with repeatable, machine-checkable criteria.
- 2.
- 3.
- 4.

SUMMARY

1. **If it matters, evaluate it** with repeatable, machine-checkable criteria.
2. **Golden datasets** anchor quality across prompt and model changes over time.
- 3.
- 4.

SUMMARY

1. **If it matters, evaluate** it with repeatable, machine-checkable criteria.
2. **Golden datasets** anchor quality across prompt and model changes over time.
3. **Traces and evals together** improve debugging speed and governance evidence.
- 4.

SUMMARY

1. **If it matters, evaluate it** with repeatable, machine-checkable criteria.
2. **Golden datasets** anchor quality across prompt and model changes over time.
3. **Traces and evals together** improve debugging speed and governance evidence.
4. **Release confidence** comes from measurable evidence, not manual review alone.

BRIDGE TO MODULE 7

What we have now:

-

What is next:

-

-



BRIDGE TO MODULE 7

What we have now:

- A repeatable eval suite that produces evidence-based release recommendations.

What is next:

-
-



BRIDGE TO MODULE 7

What we have now:

- A repeatable eval suite that produces evidence-based release recommendations.

What is next:

- Evaluation results need to gate deployment, not just inform it.
-



BRIDGE TO MODULE 7

What we have now:

- A repeatable eval suite that produces evidence-based release recommendations.

What is next:

- Evaluation results need to gate deployment, not just inform it.
- Your first task in Module 7: define which eval criteria become hard CI gates before a release can proceed.



BRIDGE TO MODULE 7

What we have now:

- A repeatable eval suite that produces evidence-based release recommendations.

What is next:

- Evaluation results need to gate deployment, not just inform it.
- Your first task in Module 7: define which eval criteria become hard CI gates before a release can proceed.

Module 7 covers deployment, observability, and governance: shipping AI features safely and maintaining control after

release.

QUESTIONS?

Module 6 — Governed AI Feature Delivery